

Module 1

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Logic

- use two loops
 1. for row and 1 for column
- Outer loop print the rows
- Inner loop print the column
in this inner loop print * give time

Algorithm

1. Start
2. Input n
3. Repeat from $i=1$ to n
4. Repeat from $j=1$ to n
5. Print *
6. Print a new line
7. Stop

Time Complexity

- Outer loop = n
- Inner loop = n
- $O(n^2)$

Space Complexity

$O(1)$

Explanation

There are n rows and n columns. Every position is printed once, resulting in $n \times n$ operations. Only loop variables are used, so that extra memory remains constant.

Optimization

No better algorithm exists because every character must be printed.

Code:- `import java.util.Scanner`

`class Square`

`public static void main (String[] args) {`

`Scanner sc = new Scanner (System.in)`

`int n = sc.nextInt();` (n=5)

`for (int i=1; i <= n; i++) {`

`for (int j=1; j <= n; j++) {`

`System.out.print ("*");` } }

`System.out.println();` } }

Square.java 1

Module_1 > Square.java > Square > main(String[])

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class Square {
6
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10        int N = sc.nextInt();
11
12        for (int i = 1; i <= N; i++) {
13            for (int j = 1; j <= N; j++) {
14                System.out.print(s: "");
15            }
16            System.out.println();
17        }
18    }
19 }
20 }
21 }
```

Run | Debug

powershell X

PS C:\Users\Solv-IT\Desktop\java_module> javac Module_1/Square.java

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/Square

5

PS C:\Users\Solv-IT\Desktop\java_module>

2. * * * * *

 * *

 * *

 * *

 * * * * *

Logic

Algorithm

1. Read n
2. Run ~~outer~~ outer loop for rows
3. Run inner loop for columns.
4. Print "*" based on condition

5. Print a new line

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation:

Every row and column is visited once. Boundary cells print '*', while inner cells print spaces. The algorithm still checks every position so that total work is proportional to n^2 .

Optimization

Note possible since every cell must be processed.

Code:-

```
for(int i=1; i<=n; i++) {
```

($n=5$)

```
  for(int j=1; j<=n; j++) {
```

```
    if(i==1 || j==1 || i==n || j==n) {
```

```
      System.out.print("*");
```

```
    } else {
```

```
      System.out.print(" ");
```

```
    }
```

```
  }
  System.out.println();
```

Module_1 > J HollowSquare.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class HollowSquare {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= N; j++) {
15                 if (i == 1 || j == 1 || i == N || j
16                     == N) {
17                     System.out.print(s: "*");
18                 } else {
19                     System.out.print(s: " ");
20                 }
21             }
22             System.out.println();
23         }
24     }
25 }
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/HollowSquar
5
*****
*   *
*   *
*   *
*****
PS C:\Users\Solv-IT\Desktop\java_module>
```

3) Triangle

```
*
* *
* * *
* * * *
* * * * *
```

Algorithm

1. Read n .
2. Run outer loop for rows
3. Run inner loop for columns upto less or equal to outer loop
4. Print "*"
5. Move to next line

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

The first row prints one star, the second prints two and so on up to n stars. The total number of prints is $1+2+\dots+n = n(n+1)/2$, which simplifies to $O(n^2)$.

Optimization

Cannot be improved because each output character must be printed

Code:

```
for (int i=1; i<=n; i++) {
    for (int j=1; j<=i; j++) {
        System.out.print("*");
        System.out.print("\n");
    }
}
```

($n=5$)

Module_1 > Triangle.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class Triangle {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= i; j++) {
15                 System.out.print(s: "");
16             }
17             System.out.println();
18         }
19     }
20 }
21 }
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/Triangle
5
*
**
***
****
*****
PS C:\Users\Solv-IT\Desktop\java_module>
```

4. Inverted Triangle

```
* * * * *  
* * * *  
* * *  
* *  
*
```

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

The first row prints n stars, then $n-1$, down to 1. The total work is again $n(n+1)/2$, giving quadratic time.

Optimization

No optimization beyond printing fewer characters, which would change the required output.

Code

```
for (int i = n; i >= 1; i--) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print(" * ");  
    }  
    System.out.println();  
}
```

Module_1 > InvertedTriangle.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class InvertedTriangle {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = N; i >= 1; i--) {
14             for (int j = 1; j <= i; j++) {
15                 System.out.print(" ");
16             }
17             System.out.println();
18         }
19     }
20 }
21
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/InvertedT
gle
5
*****
****
***
**
*
PS C:\Users\Solv-IT\Desktop\java_module>
```

S. Pyramid

```
  * * *
 * * * * *
* * * * * *
* * * * * * *
```

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

Each row prints spaces followed by stars. Then total number of printed characters grows proportionally to n^2 .

Optimization

Not possible because the full pattern must be generated.

Code

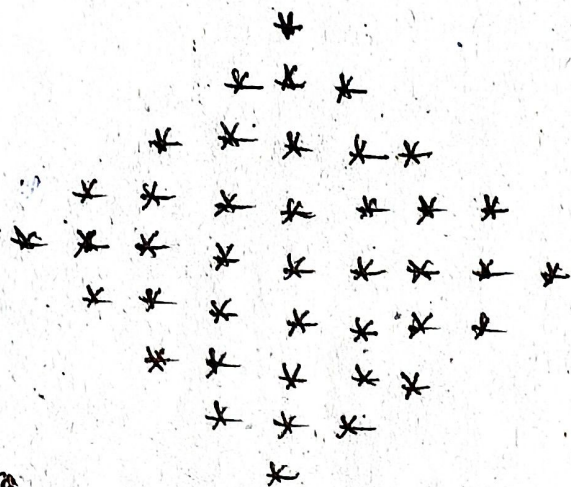
```
for (int i = 1; i <= n; i++) {
    for (int j = n - i; j >= 1; j--) {
        System.out.print(" ");
    }
    for (int k = 1; k <= 2 * i - 1; k++) {
        System.out.print("*");
    }
    System.out.println();
}
```

$n = 5$

▶ + ~ ◀ 🔒

```
PS C:\Users\Solv-IT\Desktop\java module>
```

Diamond



Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

The upper and lower halves each require roughly $n^2/2$ operations. Combined, the complexity remains $O(n^2)$.

Optimization

No optimized approach exists for full pattern printing

Code

```

for(int i = 1; i <= n; i++) {
    for(int j = n-1; j >= i; j--) {
        System.out.print(" ");
    }
    for(int k = 1; k <= 2*i-1; k++) {
        System.out.print("*");
    }
    System.out.println();
    for(int i = n-1; i >= 1; i--) {
        for(int j = n-1; j >= i; j--) {
            System.out.print(" ");
        }
        for(int k = 1; k <= 2*i-1; k++) {
            System.out.print("*");
        }
        System.out.println();
    }
}

```

Module_1 > J Diamond.java > Diamond > main(String[])

```

1 package Module_1;
2 import java.util.Scanner;
3 public class Diamond {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7         int N = sc.nextInt();
8
9         for (int i = 1; i <= N; i++) {
10
11             for (int j = N - 1; j >= i; j--) {
12                 System.out.print(s: " ");
13             }
14             for (int j = 1; j <= 2 * i - 1; j++) {
15                 System.out.print(s: "*");
16             }
17             System.out.println();
18         }
19
20         for (int i = N - 1; i >= 1; i--) {
21
22             for (int j = N - 1; j >= i; j--) {
23                 System.out.print(s: " ");
24             }
25             for (int j = 1; j <= 2 * i - 1; j++) {
26                 System.out.print(s: "*");
27             }
28             System.out.println();
29         }
30     }
31 }
32

```

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/Diamond

```

5
*
***
*****
*****
*****
*****
***
*

```

PS C:\Users\Solv-IT\Desktop\java_module>

7) Half Diamond

```
*  
* *  
* * *  
* * * *  
* * * * *  
* * * *  
* * *  
* *  
*
```

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

The pattern first increase and then decreases in size.
The total printed characters are proportional to n^2 .

Optimization

Not applicable

Code:

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=i; j++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
    for(int i=n-1; i>=1; i--) {  
        for(int j=1; j<=i; j++) {  
            System.out.print(" * ");  
        }  
        System.out.println();  
    }  
}
```

(n=5)

Module_1 > HalfDiamond2.java > HalfDiamond2 > main(String[] args)

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class HalfDiamond2 {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= i; j++) {
15                 System.out.print(s: "");
16             }
17
18             System.out.println();
19         }
20
21         for (int i = N - 1; i >= 1; i--) {
22
23             for (int j = 1; j <= i; j++) {
24                 System.out.print(s: "");
25             }
26
27             System.out.println();
28         }
29
30     }
31 }
32
```

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/HalfDiamond2

```
5
*
**
***
****
*****
****
***
**
```

Focus folder in explorer (ctrl + click)

PS C:\Users\Solv-IT\Desktop\java_module>

8. Half Diamond Inverted

```
* * * * *
 * * * *
  * * *
   * *
    *
```

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

The algorithm prints decreasing row of stars, processing all required positions. The number of operations grows quadratically.

Optimization

Not possible

Code: —

```
for(int i=n; i>=1; i--) {
    for(int j=n-1; j>=1; j--) {
        System.out.print(" ");
    }
    for(int k=1; k<=i; k++) {
        System.out.print("* ");
    }
    System.out.print("\n");
}
```

(n=5)

Module_1 > J HalfDiamondInverted.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class HalfDiamondInverted {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = N; i >= 1; i--) {
14
15             for (int j = N - 1; j >= i; j--) {
16                 System.out.print(" ");
17             }
18             for (int j = 1; j <= i; j++) {
19                 System.out.print("* ");
20             }
21             System.out.println();
22         }
23     }
24 }
25
26 }
```

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1\HalfDiamondInverted

5

**

*

PS C:\Users\Solv-IT\Desktop\java_module>

Number Patterns

Square Numbers

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Explanation

Every row contains n numbers, and there are n rows.
Thus, $n \times n$ values are printed.

Optimization

None

Code:

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=n; j++) {  
        System.out.print("1");  
        System.out.print("\n");  
    }  
}
```

(n=5)

Module_1 > J SquareNumber.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class SquareNumber {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= N; j++) {
15                 System.out.print("1");
16             }
17             System.out.println();
18         }
19     }
20 }
21
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/SquareNumber
5
11111
11111
11111
11111
11111
PS C:\Users\Solv-IT\Desktop\java_module>
```

Increasing Numbers

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

Each row prints n numbers in increasing order. The total output is n^2 numbers.

Optimization

None

Code

```
for(int i=1; i<=n; i++) {
```

```
    for(int j=1; j<=n; j++) {
```

```
        System.out.print(j);
```

```
    }  
    System.out.println();
```

($n=5$)

IncreasingSquareNumber.java 1

Module_1 > IncreasingSquareNumber.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class IncreasingSquareNumber {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= N; j++) {
15                 System.out.print(j);
16             }
17             System.out.println();
18         }
19     }
20 }
21
```

powershell X

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/IncreasingSquareNumber
5
12345
12345
12345
12345
12345
PS C:\Users\Solv-IT\Desktop\java_module>
```

3. Right Triangle

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

Rows increase from 1 to n numbers. The total printed value equal $n(n+1)/2$, gives $O(n^2)$

Optimization

Note

Code

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print(j);  
    }  
    System.out.println();  
}
```

(n=5)

NumberTriangle.java 1

Module_1 > J NumberTriangle.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class NumberTriangle {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = 1; i <= N; i++) {
14             for (int j = 1; j <= i; j++) {
15                 System.out.print(j);
16             }
17             System.out.println();
18         }
19     }
20 }
21
```

... powershell X

```
PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/NumberTriangle
5
1
12
123
1234
12345
PS C:\Users\Solv-IT\Desktop\java_module>
```

4. Inverted Diamond

3 3 3 3 3

4 4 4 4

5 5 5

6 6

7

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

The width decreases row by row, but the total output is still proportional to n^2 .

Optimization

None

Code

```
for (int i = n; i >= 1; i--)  
{  
    for (int j = n-1; j >= i; j--)  
        System.out.print(" ");  
    for (int j = 1; j <= i; j++)  
        System.out.print((n-i)+3);  
    System.out.print("\n");  
}
```

(n=5)

Module_1 > IncreasingInvertedDiamond.java > {} Module_1

```
1 package Module_1;
2
3 import java.util.Scanner;
4
5 public class IncreasingInvertedDiamond {
6
7     Run | Debug
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = N; i >= 1; i--) {
14
15             for (int j = N - 1; j >= i; j--) {
16                 System.out.print(" ");
17             }
18             for (int j = 1; j <= i; j++) {
19                 System.out.print(N - i + 3 + " ");
20             }
21             System.out.println();
22         }
23     }
24 }
25
26 }
```

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/IncreasingInvertedDiamond

```
5
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

PS C:\Users\Solv-IT\Desktop\java_module>

Sandwich Pattern

```
3 3 3 3 3
  4 4 4
    5
  4 4 4
3 3 3 3 3
```

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

The algorithm prints both spaces and numbers for each row. All positions are processed, resulting in quadratic time.

Optimization
None

code

```
for (int i = n; i >= 1; i--) {
    for (int j = n - 1; j >= i; j--) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print(n - i + 3);
    }
    System.out.println();
    for (int i = 2; i <= n; i++) {
        for (int j = n - 1; j >= i; j--) {
            System.out.print(" ");
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            System.out.print(n - i + 3);
        }
        System.out.println();
    }
}
```

$n=3$

Module 1 > J Sandwich.java > {} Module 1

```

1  package Module_1;
2
3  import java.util.Scanner;
4
5  public class Sandwich {
6
7      fun(Debug
8      public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         int N = sc.nextInt();
12
13         for (int i = N; i >= 1; i--) {
14
15             for (int j = N - 1; j >= i; j--) {
16                 System.out.print(s: " ");
17             }
18             for (int j = 1; j <= 2 * i - 1; j++) {
19                 System.out.print(N - i + 3);
20             }
21
22             System.out.println();
23         }
24
25         for (int i = 2; i <= N; i++) {
26
27             for (int j = N - 1; j >= i; j--) {
28                 System.out.print(s: " ");
29             }
30             for (int j = 1; j <= 2 * i - 1; j++) {
31                 System.out.print(N - i + 3);
32             }
33
34             System.out.println();
35         }
36     }
37 }
38

```

```

PS C:\Users\Solv-IT\Desktop\java_module> java Module_1/Sandwich
3
33333
333
5
33333
PS C:\Users\Solv-IT\Desktop\java_module>

```

Increasing Diamond Pattern

```
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3
```

Time Complexity
 $O(n^2)$

Space Complexity
 $O(1)$

Explanation

The pattern contains an upper and lower half. The total no. of printed elements is proportional to n^2 .

Optimization

None

Code

```
for(int i=1; i<=n; i++){ (n=5)
    for(int j=1; j<=i; j++){
        System.out.print((n+i)-3);
    }
    System.out.println();
}
for(int i=n-1; i>=1; i--){
    for(int j=1; j<=i; j++){
        System.out.print((n-i+1)-3);
    }
    System.out.println();
}
```

```
PS C:\Users\Solv-IT\Desktop\java module> java Module 1/IncreasingHalfDiamond
```

```
PS C:\Users\Solv-IT\Desktop\java_module>
```